

Estimasi Nilai π Menggunakan Monte Carlo Simulation dengan OpenMP

Mata Kuliah: Komputasi Paralel dan Terdistribusi — Semester Genap 2024/2025

Institusi: Universitas Palangkaraya — Teknik Informatika

Tim

Nama	NIM	Peran
Narsa Elvanisa	2430305030032	Project Manager / System Architect
Nailah Nur Fadila	2430205030030	Core Developer
Gabriella Chantika Agnes Kristin	2430205030025	Core Developer
Muhammad Aziz Arrahman	2430305030020	QA & Dokumentasi

Deskripsi Proyek

Program ini mengimplementasikan **Monte Carlo Simulation** untuk mengestimasi nilai π (ϕ) menggunakan paralelisme **OpenMP** pada arsitektur multi-core CPU.

Prinsip Kerja:

- Bangkitkan N titik acak (x, y) dalam kotak $[0,1] \times [0,1]$
- Hitung berapa titik yang jatuh di dalam lingkaran satuan ($x^2 + y^2 \leq 1$)
- $\pi \approx 4 \times (\text{titik dalam lingkaran} / N)$

- Semakin besar $N \rightarrow$ semakin akurat estimasi π
-

Prerequisites

```
OS      : Linux (Ubuntu 20.04+) atau Windows dengan WSL2
Compiler: GCC 9+ dengan dukungan OpenMP
        → sudo apt install gcc
        → Cek: gcc --version
Python  : 3.8+ dengan matplotlib (untuk grafik)
        → pip install matplotlib
RAM      : Minimal 2 GB (untuk N = 100 juta)
```

Cara Instalasi & Menjalankan

1. Clone / Download Repositori

```
bash

git clone https://github.com/[USERNAME]/[REPO_NAME].git
cd [REPO_NAME]
```

2. Build Semua Binary

```
bash

make all
```

3. Jalankan Sequential Baseline

```
bash  
  
make run_seq  
# atau custom N:  
./monte_carlo_seq 50000000
```

4. Jalankan Versi Paralel OpenMP

```
bash  
  
make run_par  
# atau custom N dan T:  
./monte_carlo_par 10000000 4  
#                ↑N=10 juta ↑T=4 thread
```

5. Uji Semua Konfigurasi Thread Sekaligus

```
bash  
  
make run_all_threads
```

6. Jalankan Unit Test (8 test)

```
bash  
  
make test
```

Output yang diharapkan:

```
[PASS] Output dalam rentang wajar (2.0-5.0)
[PASS] Hasil paralel mendekati sequential
[PASS] N=1 menghasilkan 0.0 atau 4.0
[PASS] N=10 juta, T=4: error < 0.05
[PASS] Dua run identik (seed deterministik)
[PASS] Error < 0.05 untuk semua thread config
[PASS] Error menurun seiring N bertambah
[PASS] N=10 juta: error relatif < 0.1%
```

Hasil: 8/8 PASS ✓ Semua test berhasil!

7. Jalankan Benchmark Lengkap

```
bash

make benchmark
# → Hasil: results/benchmark_results.csv
```

8. Buat Grafik Performa

```
bash

make analyze
# → Hasil: results/grafik_speedup.png
#           results/grafik_efisiensi.png
#           results/grafik_waktu.png
```

Struktur Direktori

```
.
├─ monte_carlo_seq.c      # Sequential baseline (PIC: Gabriella)
├─ monte_carlo_par.c      # Parallel OpenMP (PIC: Nailah)
├─ benchmark.c            # Benchmark semua konfigurasi (PIC: Gabriella)
├─ test_correctness.c     # 8 unit test correctness (PIC: Nailah)
├─ analyze.py             # Grafik speedup & efisiensi (PIC: Aziz)
├─ Makefile               # Build system
├─ README.md              # Dokumen ini
└─ results/               # Output benchmark dan grafik (auto-generated)
    ├─ benchmark_results.csv
    ├─ grafik_speedup.png
    ├─ grafik_efisiensi.png
    └─ grafik_waktu.png
```

Metrik Performa

Metrik	Formula	Target
Speedup $S(p)$	$T_{seq} / T_{par}(p)$	$\geq 2.5\times$ pada $p=4$
Efisiensi $E(p)$	$S(p) / p$	$\geq 60\%$ pada $p=4$
Error π	$ \pi_{est} - \pi_{ref} / \pi_{ref} \times 100\%$	$< 0.1\%$ untuk $N=10M$

Git Tags

Tag	Milestone
v1.0-week1	Proposal & tim
v2.0-week2	SDD & desain arsitektur
v3.0-week3	Implementasi inti & unit test
v4.0-week4	Benchmark & analisis performa
v5.0-final	Laporan akhir & presentasi

Lisensi

MIT License — bebas digunakan untuk tujuan akademik.